

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Answers:

1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering process, not random execution. The team should first analyze requirements, identify risks, design test cases, define expected results, execute tests, and document defects. Structured techniques such as equivalence partitioning and boundary value analysis provide better coverage than random testing. For example, instead of randomly testing an age field, the tester can test valid, invalid, minimum, and maximum values. This improves consistency, traceability, defect detection, and software quality.

2. Well-Defined Testing Process

A well-defined testing process provides a systematic sequence such as planning, requirement analysis, test design, execution, defect reporting, retesting, and closure. In a banking system, transaction requirements can be converted into test cases for deposits, withdrawals, transfers, and invalid transactions. Each defect is documented and tracked until resolution. This reduces missed defects, improves reliability, provides measurable results, and ensures important banking functions are thoroughly tested.

3. Testing as a Structured SDLC Process

Testing should be performed throughout the SDLC rather than only after development. During requirements, testers review requirements; during design, they identify test conditions; during coding, unit testing is performed; during integration, interfaces are tested; and during system testing, complete functionality is verified. For an e-commerce system, payment, cart, login, and order modules should be tested individually and together. Process discipline helps detect defects early and prevents integration and system-level failures.

4. Error, Defect and Failure

An error is a human mistake made during development. A defect is the flaw introduced into the software because of that error. A failure occurs when the software executes and produces an incorrect result. For example, a developer may incorrectly write a login condition, which creates a defect. When a valid user is rejected during execution, it becomes a failure. Thus, Error → Defect → Failure represents the relationship between these terms.

5. Early Testing Principle

The principle “Testing should start early” helps detect defects before they become expensive to fix. Testers should review requirements and design documents before coding begins. For example, if a requirement incorrectly states the payment limit, the tester can identify it during requirement review instead of discovering it after deployment. Early detection reduces rework, development cost, and defect leakage while improving overall testing effectiveness.

6. Tester Involvement in Requirements

Testers can participate in requirement reviews to identify ambiguity, incompleteness, inconsistency, and missing conditions. For a healthcare system, a requirement such as “patient records should be updated quickly” is ambiguous because “quickly” is not measurable. The tester

can request a specific response-time requirement. Early tester involvement improves requirement quality and prevents defects from being carried into design and implementation.

7. Defects from Ambiguous Requirements

Defects can originate from unclear requirements because developers may interpret the same requirement differently. For example, if a payroll requirement does not clearly define overtime calculation, developers may implement an incorrect formula, producing wrong salaries. Clear, measurable, and unambiguous requirements reduce misunderstanding. Requirement reviews, prototypes, acceptance criteria, and tester participation can prevent such defects early in the lifecycle.

8. Boundary Value Analysis

A crash occurring at boundary input values indicates a boundary-related functional defect. Such defects commonly occur at minimum, maximum, just below, and just above valid limits. For example, if an age field accepts 18–60, the tester should test 17, 18, 19, 59, 60, and 61. Boundary Value Analysis would expose incorrect boundary handling before release and improve test coverage.

9. Defect Repository

A defect repository stores information such as defect description, module, severity, priority, root cause, status, and resolution. Testers can analyze historical defects to identify frequently failing modules and common error patterns. For example, if payment defects repeatedly occur due to invalid input handling, new test cases should specifically target those conditions. Thus, a defect repository supports better test design, prevents repeated defects, and improves software quality.

10. Functional Defect in ATM

An ATM withdrawal failure caused by incorrect transaction logic is a functional defect because the system does not perform the required operation correctly. Testers could prevent it by creating test cases for valid withdrawals, insufficient balance, invalid amounts, daily limits, and concurrent transactions. Boundary and negative testing would also help. Proper functional testing ensures that withdrawal processing follows the specified business rules.

11. Defect Clustering

Defect clustering means that a small number of modules often contain a large proportion of defects. The testing team should analyze defect history and identify high-defect modules. For example, if the payment and authentication modules contain most defects, these modules should receive more detailed testing. This helps prioritize testing resources, increases defect detection efficiency, and reduces the risk of critical failures.

12. Performance Defect

Slow performance under heavy user load is classified as a performance/non-functional defect. It can be identified through load, stress, and scalability testing. For example, if a web application becomes extremely slow when 10,000 users access it simultaneously, the issue may involve database queries, server capacity, or inefficient code. Proper classification helps developers focus on the correct technical cause and apply suitable solutions.

13. Historical Defect Data for Regression Testing

Historical defect data can be reused to identify modules and scenarios that previously failed. These test cases should be included in the regression test suite after every release. For example, if previous releases had payment and login defects, those scenarios should always be retested after changes. Reusing historical tests improves coverage, saves test-design time, and reduces the possibility of previously fixed defects returning.

14. Tester Role in Integration Defects

Testers help minimize integration defects by participating early in requirement and design discussions and understanding interfaces between modules. They can identify unclear data formats, dependencies, and communication gaps before integration testing. For example, a tester may identify that the payment module expects an amount in rupees while another module sends it in paise. Early involvement prevents such mismatches and improves collaboration between developers and testers.

15. Defect Prevention

Defect prevention focuses on identifying and removing the root causes of recurring defects rather than repeatedly fixing individual failures. The team can use root-cause analysis, code reviews, requirement reviews, checklists, and process improvements. For example, if repeated database errors occur because developers misunderstand database standards, training and coding guidelines can prevent them. Prevention reduces recurring defects, maintenance effort, and overall development cost.

16. Structured Testing for Mobile Applications

A mobile application should use a standardized testing strategy covering requirements, test planning, functional testing, usability, compatibility, performance, and regression testing. Test cases should be documented and executed consistently across devices and operating systems. For example, the application should be tested on different screen sizes, Android versions, network conditions, and orientations. Structured testing improves reliability and reduces inconsistent results.

17. Testing vs Debugging

Testing identifies whether software contains defects, while debugging locates the cause of a defect and fixes the code. For example, testing may discover that an ATM deducts money without dispensing cash. The developer then debugs the transaction code to identify and correct the cause. After the fix, testing is performed again to verify the solution. Therefore, testing detects problems, while debugging diagnoses and removes them.

18. Cross-Browser Testing

Cross-browser testing verifies that a web application works correctly across different browsers, versions, operating systems, and screen sizes. The team should prepare test cases for major browsers such as Chrome, Firefox, Edge, and Safari. Functional, UI, compatibility, and responsive behavior should be checked. For example, if a payment button works in Chrome but not Firefox, the defect can be identified and corrected before affecting customers.

19. Proper Validation

Improper validation allows incorrect or unexpected data to enter the system. Testers should verify input validation using valid, invalid, boundary, empty, and unexpected values. For example, a logistics application accepting a negative package weight indicates inadequate validation. Test cases should ensure invalid values are rejected with appropriate messages. Proper validation prevents incorrect processing and improves system reliability.

20. Systematic Test Design

Random testing alone cannot guarantee adequate coverage. Testing as an engineering activity requires requirements analysis, test conditions, systematic test-case design, execution, and defect tracking. Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state-transition testing can be used. For example, a login system should systematically test valid credentials, invalid credentials, locked accounts, empty fields, and boundary conditions. This increases defect detection and test coverage.

21. Data Consistency in Integration

Data inconsistency between healthcare modules may result from incorrect interfaces, data mapping, synchronization, or database transactions. Testers should perform integration tests to verify that data transferred from one module is correctly received and stored by another. For example, a patient's updated phone number in the registration module should appear correctly in the appointment module. Proper integration testing ensures accurate and consistent data flow.

22. Tester Role Across SDLC

Testers should participate throughout the SDLC. In the requirements phase, they review requirements; during design, they identify risks; during development, they support test preparation and unit-test review; during integration, they test interfaces; and during system testing, they validate complete functionality. Early involvement identifies defects before coding and reduces rework, delays, and project cost.

23. Peak Usage Failure

Failure during peak usage indicates insufficient performance and test coverage. The team should identify critical scenarios and perform load, stress, and scalability testing. For example, an online ticket system should be tested with a large number of simultaneous users before a major booking event. Better coverage of peak-load scenarios helps identify capacity limitations early and prevents production failures.

24. Defect Documentation and Repository

A defect repository should record each defect with a unique ID, description, steps to reproduce, expected result, actual result, severity, priority, environment, status, and resolution. This allows developers and testers to track defects from discovery to closure. Historical records can also be analyzed for recurring patterns. Proper documentation improves traceability, communication, accountability, and defect management.

25. Structured Testing for Banking Systems

Banking systems require structured testing because transaction failures can cause serious financial impact. The team should follow requirement analysis, risk-based test planning, functional testing, integration testing, security testing, performance testing, and regression testing. Transactions such as deposits, withdrawals, transfers, invalid amounts, and insufficient balances should be tested. Structured testing improves reliability, reduces transaction failures, and increases customer trust.

26. Edge Case and Boundary Testing

Testing only normal inputs can miss failures at extreme conditions. Testers should include minimum, maximum, just below, and just above boundary values, along with invalid and unusual inputs. For example, if a transfer limit is ₹1–₹50,000, values such as ₹0, ₹1, ₹2, ₹49,999, ₹50,000, and ₹50,001 should be tested. Boundary testing improves defect detection at critical limits

27. Process-Based Testing

A process-based testing approach defines standardized activities, procedures, templates, responsibilities, and quality criteria across all modules. Every module should follow the same basic process for planning, test-case design, execution, defect reporting, and regression testing. This reduces inconsistency and ensures that one module is not tested more thoroughly merely because of individual tester preferences. Standardization produces more predictable and consistent quality.

28. Input Validation Testing

Unexpected inputs should be tested using invalid data, special characters, empty values, incorrect formats, extreme values, and unexpected combinations. For example, a registration form should reject invalid email addresses, empty mandatory fields, and unsupported characters. Validation rules should be checked against requirements. Proper input testing improves robustness, prevents crashes, and protects the application from incorrect data.

29. Defect Leakage

Defect leakage occurs when defects that should have been detected during testing reach production. Common causes include inadequate test coverage, unclear requirements, insufficient regression testing, and poor defect analysis. The team should perform root-cause analysis, strengthen test cases, conduct regression testing, and involve testers early. These practices reduce production defects and improve customer satisfaction.

30. Early Defect Detection

The team should adopt early testing, risk-based testing, systematic test design, requirement reviews, and continuous regression testing. Defects found during requirements or design are generally easier and cheaper to fix than defects discovered after deployment. For example, detecting an incorrect payment rule during requirement review avoids coding, integration, and deployment rework. Early defect detection reduces cost and effort while improving software quality.



Code of Conduct:

I Yamini C G 192412054 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Yamini C G

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand